

CS880: Final Project Report

Ansh Aggarwal Rohan Chakravarthi Yijun Zhu

Abstract

Error detection and correction (EDAC) schemes are techniques used to detect, mitigate, and correct errors over inconsistent and unreliable communication channels. Often, the number of errors that can be introduced through these channels are considered to be unbounded, however, with the advent of modern communication systems, this outlook may be considered pessimistic. As such, it is worth looking into EDAC schemes that surpass information-theoretic limits for errors that are computationally bounded. The work of Jad Silbak and Daniel Wichs across two papers: *Detecting and Correcting Computationally Bounded Errors: A Simple Construction Under Minimal Assumptions* [2024] and *Binary Codes for Error Detection and Correction in a Computationally Bounded World* [2025] explores constructions for EDAC schemes under the aforementioned setting. We highlight and explore the findings and constructions from Silbak and Wichs' work in this survey.

Contents

Abstract	1
Introduction	3
Notation and EDAC Basics	3
Alphabet	3
Hamming Distance	3
Code Syntax	3
Rate and Error Tolerance	3
Communication Channels	4
Singleton Bound	4
Blok-Zyablov Bound	4
Primitives	4
One Way Functions	4
Universal One Way Hash Functions	4
Collision Resistant Hash Functions	5
Correlation Intractability Hashing	5
Random Oracle Model	6
Target 2-Input Correlation Intractability Hashing	6
The Dark Matter Hash Assumption	7
Exploring the 2024 Paper	8
Randomized and Self-Seeded Codes	8
Randomized Codes	8
Self-Seeded Codes	9
Construction	9
Sub-Exponential Alphabet Detection	10
Constant Alphabet Detection	10
Error Correction	11
Correctness	11
Exploring the 2025 Paper	11
Selective Security vs. Adaptive Security	11
Selective Security	12
Adaptive Security	12
Construction	12
Selectively Secure Error Detection (SSED)	12
Selectively Secure Error Correction (SSEC)	12
Adaptively Secure Error Detection (ASED)	13
Adaptively Secure Error Correction (ASEC)	14
Correctness	14
Conclusion	14
Acknowledgments	15

Introduction

The two objects that fall under EDAC are error detection codes (EDC) and error correction codes (ECC) with two different objectives. Consider the following setup: let $\mathcal{S}$ be the sender and let $\mathcal{R}$ be the receiver, and let m_s be the message that $\mathcal{S}$ would like to send to $\mathcal{R}$ and let m' be the message that $\mathcal{R}$ receives.

1. The goal of error detection is to allow $\mathcal{R}$ to verify whether $m = m'$
2. The goal of error correction is to allow $\mathcal{R}$ to recover m from m' even if $m \neq m'$

Notation and EDAC Basics

Alphabet

The alphabet for a code refers to the set of symbols that may be used to construct the code. This set is represented with $[q]$ where q is number of symbols in that set. Note that if the alphabet is simply $\{0, 1\}$, then the code is called a binary code.

Hamming Distance

The central metric for EDAC schemes is Hamming distance: for $x, y \in [q]^n$, we have

$$\Delta(x, y) = |\{i \mid x_i \neq y_i\}| \quad (1)$$

Another notion is the relative Hamming distance which is simply $\delta(x, y) = \Delta(x, y)/n$. For example, for the words *cat*, *dog*, and *god* in the standard English alphabet, we have $\Delta(\text{cat}, \text{dog}) = 3$, $\Delta(\text{cat}, \text{god}) = 3$, and $\Delta(\text{god}, \text{dog}) = 2$.

Code Syntax

Let k, n, q be parameters and let **Encode**: $\{0, 1\}^k \rightarrow [q]^n$ be a function. **Encode** is an encoding function that:

1. Decode d errors: If $\exists$ **Decode** : $[q]^n \rightarrow \{0, 1\}^k$ such that for $m \in \{0, 1\}^k$ and $v \in [q]^n$, if $\Delta(\text{Encode}(m), v) \leq d$, then **Decode**(v) = m
2. Detect d errors: If $\exists$ **Decode** : $[q]^n \rightarrow \{\perp\} \cup \{0, 1\}^k$ such that for $m \in \{0, 1\}^k$ and $v \in [q]^n$, and **Encode**(m) $\neq v$ but $\Delta(\text{Encode}(m), v) \leq d$, then **Decode**(v) = $\perp$

Additionally, some codes include a notion of L -list-decodes where **Decode** is allowed to output up to L values and for $m \in \{0, 1\}^k$ and $v \in [q]^n$, if $\Delta(\text{Encode}(m), v) \leq d$, then $m \in \text{Decode}(v)$. In essence, this means the decode function outputs a list of potential decodes instead of outputting a single decode for a given codeword.

Rate and Error Tolerance

EDAC schemes are assessed primarily on the basis of two parameters:

1. The rate of the code is R where $R = \frac{k}{n \log q}$ where k is number of message bits and n is number of redundancy / code bits and q is the size of the alphabet. Note that for binary codes, this is simply $R = k/n$

2. The relative error tolerance, ρ , of a code is the ratio of the number of errors a code can handle, d , and the number of bits in the code word, n thus, $\rho = \frac{d}{n}$; it should be noted that $\rho \leq 1/2$ for error correction and $\rho \leq 1$ for error detection

Communication Channels

Essentially, a communication channel is the medium that is used to transmit the message from $\mathcal{S}$ to $\mathcal{R}$. In the real world, this could range from carrier pigeons to using semaphores but in context of these papers (and generally EDAC schemes), we're usually referring to digital transmission methods. When EDAC schemes were studied during its pioneering days in the 1940s, the standard transmission methods were extremely lossy and inconsistent and as such, the amount of errors that communication channel could introduce during transmission was considered to be unbounded. With the introduction of more reliable communication channels, a natural modification to this assumption is to make errors bounded and as the work that we explore finds, this allows EDAC schemes to approach theoretically optimal limits, that is $\rho \approx 1$ and $\rho \approx 1/2$ for detection and correction respectively. In work related to error correcting codes, the notion of an adversarial channel is used which, as the name might suggest, picks the worst-case message for any given scheme/construction in order to make it decode a code word incorrectly. This notion is used throughout the papers and is assumed in this survey as well.

Singleton Bound

For linear codes, the Singleton bound is an upper bound for the number of codes for a given code with alphabet $[q]$ that is tighter than the trivial q^n bound. This implies that for any rate $R \geq 0$, we have that $\rho \leq 1 - R$ for detection and $\rho \leq \frac{1}{2}(1 - R)$.

Blokh-Zyablov Bound

The Blokh-Zyablov bound gives a bound for the best-known information theoretic rate R and error tolerance p for efficiently list-decodable binary codes.

Primitives

This is a generalized overview of assumptions and primitives that are used across both papers that include fairly standard assumptions like one way functions but also some far more exotic ones like *crypto dark matter*. These definitions are taken from the original papers for consistency.

One Way Functions

A probabilistic polynomial-time (PPT) algorithm $F : \{0, 1\}^* \rightarrow \{0, 1\}^*$ is a one-way function if for every PPT adversary and every sufficiently large $k \in \mathbb{N}$, we have:

$$\Pr_{x \leftarrow \{0,1\}^k} [A(1^k, F(x)) \in F^{-1}(F(x))] \leq \text{negl}(k)$$

In simple terms, given $F(x)$, it is hard for a polynomial time adversary to find x .

Universal One Way Hash Functions

For a security parameter $k \in \mathbb{N}$. A family of functions $\mathcal{H} = \{h_s : \{0, 1\}^{n(k)} \rightarrow \{0, 1\}^{l(k)}\}_{s \in \{0,1\}^{l(k)}}$ is a family of universal one way hash functions if it satisfies:

1. Efficiency: Given $s \in \{0, 1\}^{l(k)}$ and $x \in \{0, 1\}^{n(k)}$, $h_s(x)$ can be evaluated in time $\text{poly}(n(k), l(k))$
2. Shrinking: $l(k) < n(k)$
3. Target Collision Resistance: For every PPT adversary and sufficiently large $k \in \mathbb{N}$, if we consider the following randomized experiment:
 - (a) $(x, \text{state}) \leftarrow A(1^k) \in \{0, 1\}^{n(k)} \times \{0, 1\}^*$
 - (b) Sampling $s \leftarrow \{0, 1\}^{l(k)}$
 - (c) $x' \leftarrow A(\text{state}, s)$

Then $\Pr[x \neq x' \text{ and } h_s(x) = h_s(x')] \leq \text{negl}(k)$

Collision Resistant Hash Functions

For a security parameter $k \in \mathbb{N}$. A family of functions $\mathcal{H} = \{h_s : \{0, 1\}^{n(k)} \rightarrow \{0, 1\}^{l(k)}\}_{s \in \{0, 1\}^{l(k)}}$ is a family of universal one way hash functions if it satisfies:

1. Efficiency: Given $s \in \{0, 1\}^{l(k)}$ and $x \in \{0, 1\}^{n(k)}$, $h_s(x)$ can be evaluated in time $\text{poly}(n(k), l(k))$
2. Shrinking: $l(k) < n(k)$
3. Collision Resistance: For every PPT adversary and sufficiently large $k \in \mathbb{N}$,

$$\Pr_{s \leftarrow \{0, 1\}^{l(k)}; x, x' \leftarrow A(s)}[x \neq x' \text{ and } h_s(x) = h_s(x')] \leq \text{negl}(k)$$

The main difference between UOWHFs and CRHFs is the third requirement where the difference is that UOWHFs fix x and an output and demand the adversary come up with another x' such that the outputs match while collision resistance demands that the adversary come up with two distinct message that have the same output.

Correlation Intractability Hashing

Let $k \in \mathbb{N}$ be a security parameter and $t \in \text{poly}$ be a poly-time computable function. The hash family $\mathcal{H} = \{h_s : \{0, 1\}^{n(k)} \rightarrow \{0, 1\}^{m(k)}\}_{s \in \{0, 1\}^{l(k)}}$ is correlation intractable for t -size circuits (t -CI) if for every size(t) circuit family $f = \{f_k : \{0, 1\}^{n(k)} \rightarrow (\{0, 1\}^{m(k)})^{L(k)}\}_{k \in \mathbb{N}}$ whose output is interpreted as a set, and every (non-uniform) PPT adversary A :

$$\Pr_{s \leftarrow \{0, 1\}^{l(k)}; x \leftarrow A(s)}[h_s(x) \in f_k(x)] \leq \text{negl}(k)$$

In simpler terms, a hash family $\mathcal{H}$ is t -CI for a sparse relationship $R(x, y)$ if given a random seed $s \leftarrow \{0, 1\}^{l(k)}$, it is computationally hard to find an x , such that $(x, h_s(x)) \in R$. A sparse relationship, loosely, means that elements contained in it have very few or no special algebraic relationships or patterns that link them. This is not formal, but I believe this is a notion similar to the idea of perfect secrecy in that it says that given some hash output, the probability that it was computed from m is the same as the probability it was computed from some other m' .

Random Oracle Model

The Random Oracle Model (ROM) is an idealized cryptographic model in which a hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^m$ is treated as a truly random function. Formally, there exists a public oracle $\mathcal{O}_H$ such that:

$$\mathcal{O}_H(x) = \begin{cases} y, & \text{if } x \text{ has been queried before and } \mathcal{O}_H(x) = y \text{ was recorded} \\ y \xleftarrow{\$} \{0, 1\}^m, & \text{if } x \text{ is a new query} \end{cases}$$

The oracle maintains a table of previous queries and outputs. For every new query x , the output y is sampled uniformly at random from $\{0, 1\}^m$ and stored. The oracle is deterministic from the adversary's perspective: repeated queries always return the same output.

In this model, all parties—including the adversary—have oracle access to H through $\mathcal{O}_H$. The ROM enables proofs of cryptographic security under the assumption that H behaves like an ideal random function.

Target 2-Input Correlation Intractability Hashing

Definition

Let $k \in \mathbb{N}$ be a security parameter and $t \in \text{poly}$ be a poly-time computable function. The hash family $\mathcal{H} = \{h_s : \{0, 1\}^{n(k)} \rightarrow \{0, 1\}^{m(k)}\}_{s \in \{0, 1\}^{l(k)}}$ is target 2-input correlation intractable for t -size (t -T2CI) if for every $L \in \text{poly}$, size(t) circuit $f = \{f_k : \{0, 1\}^{n(k)} \times \{0, 1\}^{n(k)} \times \{0, 1\}^{m(k)} \rightarrow (\{0, 1\}^{m(k)})^{L(k)}\}_{k \in \mathbb{N}}$ and every PPT adversary A , if we consider the experiment:

1. $(x, \text{state}) \leftarrow A(1^k) \in \{0, 1\}^{n(k)} \times \{0, 1\}^*$
2. Sampling $s \leftarrow \{0, 1\}^{l(k)}$
3. $x' \leftarrow A(\text{state}, s)$

Then $\Pr[h_s(x') \in f_k(x', x, h_s(x)) \text{ and } x' \neq x] \leq \text{negl}(k)$. The main goal of t -T2CI from t -CI is that we'd like intractability to hold for two pairs of hash computations simultaneously. The authors construct T2CI from CI and UOWHFs which they build using the LWE assumption.

Construction. We construct a t -T2CI hash family $\mathcal{H}$ from two well-established cryptographic primitives:

- A correlation intractable (CI) hash family:

$$\mathcal{H}_{\text{CI}} = \{h_{s_1}^{\text{CI}} : \{0, 1\}^{n(k)} \rightarrow \{0, 1\}^{m_1(k)}\}_{s_1 \in \{0, 1\}^{\ell_1(k)}}$$

This is a family of hash functions parameterized by a seed s_1 , where each function takes an input of length $n(k)$ and maps it to an output of length $m_1(k)$. Correlation intractability ensures that it is hard for an adversary to find an input x that satisfies some circuit-defined relation $R(x, h_{s_1}^{\text{CI}}(x)) = 1$.

- A universal one-way hash family (UOWHF):

$$\mathcal{H}_{\text{UOWHF}} = \{h_{s_2}^{\text{UOWHF}} : \{0, 1\}^{n(k)} \rightarrow \{0, 1\}^{m_2(k)}\}_{s_2 \in \{0, 1\}^{\ell_2(k)}}$$

This is a family of hash functions where, even if an adversary chooses a fixed input x , it is computationally infeasible to find a different $x' \neq x$ such that $h_{s_2}^{\text{UOWHF}}(x') = h_{s_2}^{\text{UOWHF}}(x)$. The UOWHF component prevents output collisions on related or distinct inputs.

We define the composed hash function h_s in the T2CI family $\mathcal{H}$ as the concatenation of the two components:

$$\mathcal{H} = \{h_s(x) = h_{s_1}^{\text{CI}}(x) \parallel h_{s_2}^{\text{UOWHF}}(x)\}_{s=(s_1, s_2)}$$

- The overall seed s is the tuple (s_1, s_2) , combining the seeds of the two subcomponents.
- The function $h_s(x)$ outputs a string of length $m_1(k) + m_2(k)$ by concatenating the outputs of the CI and UOWHF hash functions applied to the same input x . This ensures the combined output carries both types of cryptographic hardness properties.

Step-by-step justification:

1. **Correlation intractability from h^{CI} :** The purpose of using the CI hash function is to prevent an adversary from producing inputs x that satisfy a predetermined circuit-based relation involving the hash output. This is crucial for the “*target*” aspect of T2CI: the adversary is given $(x, h_s(x))$ and must not be able to find $x' \neq x$ such that $h_s(x')$ satisfies some computable relation involving x , x' , and $h_s(x)$. Without this component, correlation-based attacks would be feasible.
2. **Second preimage resistance from h^{UOWHF} :** The UOWHF component ensures that once x is fixed, an adversary cannot find $x' \neq x$ such that $h_{s_2}^{\text{UOWHF}}(x') = h_{s_2}^{\text{UOWHF}}(x)$. This guarantees that if the adversary tries to find an x' that maps to the same output as x , such a collision is infeasible. This prevents “mimicry” attacks where x' might be constructed to match x under certain parts of the hash.
3. **Combined protection via concatenation:** The final hash output is the concatenation of the two component hashes: $h_{s_1}^{\text{CI}}(x) \parallel h_{s_2}^{\text{UOWHF}}(x)$. This ensures that any adversary would need to simultaneously:
 - satisfy the correlation intractability condition imposed by h^{CI} , **and**
 - avoid causing a collision in h^{UOWHF} for $x' \neq x$,

which is computationally infeasible under the respective assumptions. The composition makes the adversary’s job strictly harder than breaking either property individually.

Summary. This construction leverages the strengths of both CI and UOWHF hash families. The CI component guards against structured correlations between x and x' , while the UOWHF component ensures that even if a relation exists, producing a valid x' with a matching output remains computationally infeasible. The combination guarantees the t -T2CI property under standard cryptographic assumptions.

If both $\mathcal{H}_{\text{CI}}$ and $\mathcal{H}_{\text{UOWHF}}$ are secure under standard cryptographic assumptions (e.g., Learning With Errors), then the resulting hash family $\mathcal{H}$ satisfies the t -T2CI property.

The Dark Matter Hash Assumption

Let $k \in \mathbb{N}$ be a parameter and $0 < \delta$ be a constant and define $\tilde{k} = \lceil (\log_3 2)k \rceil$, $v = (1 + \delta)\tilde{k}$ and $\tilde{v} = \lceil (\log_2 3)v \rceil$. For every sufficiently small constant $0 < \beta < \delta$, and every PPT adversary A , when considering the following experiment:

1. Sample $\mathbf{B} \leftarrow \mathbb{F}_3^{\tilde{k} \times v}$ and $\mathbf{A} \leftarrow \mathbb{F}_2^{\tilde{v} \times \beta k}$ and set $s := (\mathbf{A}, \mathbf{B})$

$$2. (x, x') \leftarrow A(s)$$

It holds that:

$$\Pr[x \neq x' \text{ and } h_s(x) = h_s(x')] \leq \left(\frac{1}{2}\right)^{\beta k(1-o(1))}$$

Where we define $h_s : \{0, 1\}^k \rightarrow \{0, 1\}^{\beta k}$ via $h_s(x) := \text{map}_{3 \rightarrow 2}(\text{map}_{2 \rightarrow 3}(x) \cdot \mathbf{B}) \cdot \mathbf{A}$. This definition uses the $\text{map}_{a \rightarrow b}$. Formally, for every $a, b \in \mathbb{N}$, the authors define the function $\text{map}_{a \rightarrow b} : \mathbb{Z}_a^n \rightarrow \mathbb{Z}_b^{\lceil \log ba \rceil n}$ to be natural mapping from base a to base b .

Exploring the 2024 Paper

In the 2024 paper, Detecting and Correcting Computationally Bounded Errors: A Simple Construction Under Minimal Assumptions, the authors construct EDAC schemes for computationally bounded channels whilst achieving a scheme:

1. That does not rely on trusted setups; as noted in the paper, a lot of constructions under similar assumptions rely on a coordinated setup between $\mathcal{S}$ and $\mathcal{R}$ and this work avoids it
2. That has provable guarantees that rely on minimal and/or standard computation hardness problems; some constructions under similar assumptions rely on exotic hardness problems
3. With optimal tradeoffs between code rate and error tolerance for large alphabets (not binary)

Randomized and Self-Seeded Codes

The authors note that even under computationally bounded settings, it is not possible to go beyond the Singleton bound with deterministic codes which is why they introduce the notion of randomized/self-seeded codes.

Randomized Codes

For a message m , the code word v is computed with fresh randomness r every time, that is, $v \leftarrow \text{Encode}(m; r)$.

Theorem 1 (Randomized Codes) *Assuming one-way functions (OWFs), $\exists$ efficiently computable randomized error detection and error correction codes for arbitrary polynomial-time adversarial errors with the following parameters:*

1. *Detection: For a constant $0 < \varepsilon < 1$, there is a code over a constant-sized alphabet with $R = 1 - \varepsilon$ that detects a $\rho = 1 - \varepsilon$ fraction of errors*
2. *Correction: For a constant $0 < \varepsilon < 1$ and $0 \leq \rho < 1/2$, there is a code over a constant-sized alphabet with $R = 1 - \rho - \varepsilon$ that uniquely corrects ρ fraction of errors*

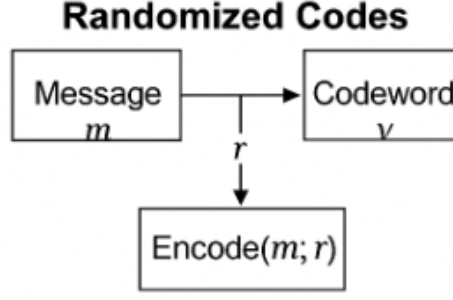


Figure 1: In randomized codes, the message is fixed first and fresh randomness is chosen each time to compute the codeword.

Self-Seeded Codes

The randomness, r , is fixed before picking a message m and thus, for arbitrary m , the code word v is computed as before, $v \leftarrow \text{Encode}(m; r)$.

Theorem 2 (Self-Seeded Codes) *Assuming collision-resistant hash functions (CRHFs), $\exists$ efficiently computable self-seeded error detection and error correction codes for arbitrary polynomial-time adversarial errors with the following parameters:*

1. *Detection: For a constant $0 < \varepsilon < 1$, there is a code over a constant-sized alphabet with $R = 1 - \varepsilon$ that detects a $\rho = 1 - \varepsilon$ fraction of errors*
2. *Correction: For a constant $0 < \varepsilon < 1$ and $0 \leq \rho < 1/2$, there is a code over a constant-sized alphabet with $R = 1 - \rho - \varepsilon$ that uniquely corrects ρ fraction of errors*

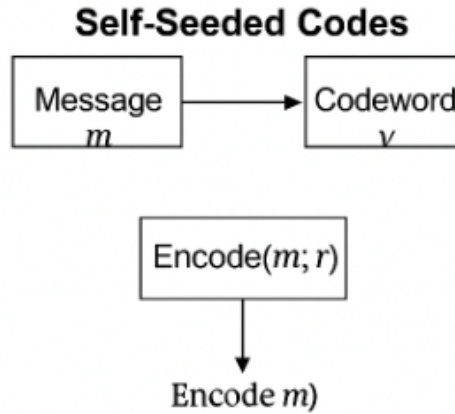


Figure 2: In self-seeded codes, the randomness (seed) is fixed first, and the codeword is computed for each message using this fixed seed.

Construction

The construction of both codes follow the same pattern and in fact, the only difference is that use of a hash function family h_r needs to a one-way function for randomized codes and a collision-resistant hash function for self-seeded codes. As such, the construction that follows is generalized for both randomized and self-seeded codes.

Sub-Exponential Alphabet Detection

Let $\mathcal{H} = \{h_r : \{0, 1\}^k \rightarrow \{0, 1\}^\lambda\}_{r \in \{0, 1\}^\lambda}$ be a family of hash functions with the desired properties. $\text{Encode}(m; r)$ creates a code c , defined as:

$$c = ([m_1, r, h_r(m)], [m_2, r, h_r(m)], \dots, [m_n, r, h_r(m)])$$

Where $m = (m_1, m_2, \dots, m_n)$. This is interpreted at the receiver's end as:

$$c' = ([m'_1, r'_1, y'_1], [m'_2, r'_2, y'_2], \dots, [m'_n, r'_n, y'_n])$$

Given c' , $\text{Decode}(c')$ does the following steps and checks:

1. Define $m' = (m'_1, m'_2, \dots, m'_n)$
2. Check if all r'_i are the same
3. Check if all y'_i are the same and check if $y'_i = h_{r'_i}(m')$

If all checks pass, then output m' otherwise output $\perp$. This scheme has a fairly large alphabet and the following scheme attempts to tackle that.

Constant Alphabet Detection

Let $(\text{Encode}_{ctrl}, \text{Decode}_{ctrl})$ be a standard error correcting code and the authors use this to define $c_{ctrl} = \text{Encode}_{ctrl}(r, h_r(m))$, effectively the second and third chunk from $(m_i, r, h_r(m))$ which was used in the previous construction but encoded. As such, $(m_i, r, h_r(m)) \mapsto (m_i, c_{ctrl})$ giving c as:

$$c = ([m_1, c_{ctrl}], [m_2, c_{ctrl}], \dots, [m_n, c_{ctrl}])$$

This is interpreted at the receiver's end as:

$$c' = ([m'_1, c'_{ctrl,1}], [m'_2, c'_{ctrl,2}], \dots, [m'_n, c'_{ctrl,n}])$$

Given c' , $\text{Decode}(c')$ follows the exact same steps as before but first, it uses Decode_{ctrl} to turn $c_{ctrl,i} \rightarrow (r'_i, y'_i)$. If any of the intermediary decodes, fail, then Decode can output $\perp$. This relies on the correctness of the control code.

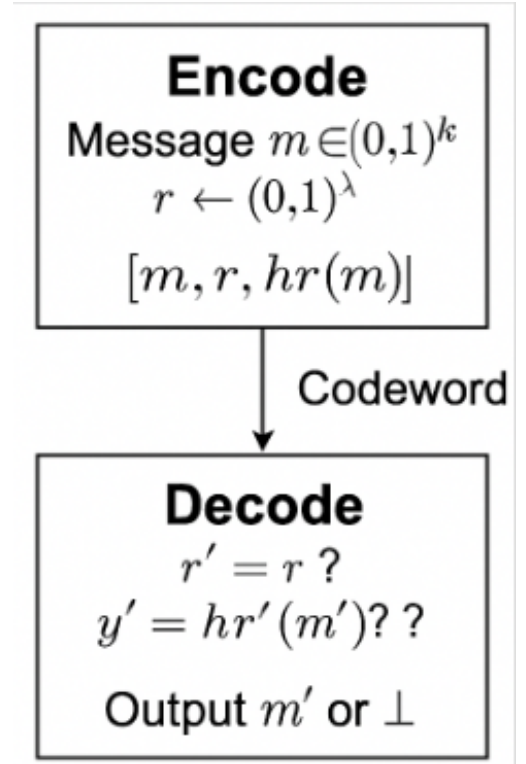


Figure 3: Constant Alphabet Detection Diagram

Error Correction

Let $(\text{Encode}_{\text{data}}, \text{Decode}_{\text{data}})$ be a list-decoding error correcting code and the authors use this to define $c_{\text{data},i} = \text{Encode}_{\text{data}}(m_i)$ which replaces m_i in the previous construction. As such, c is now defined as:

$$c = ([c_{\text{data},1}, c_{\text{ctrl}}], [c_{\text{data},2}, c_{\text{ctrl}}], \dots, [c_{\text{data},n}, c_{\text{ctrl}}])$$

This is interpreted as:

$$c' = ([c'_{\text{data},1}, c'_{\text{ctrl}}], [c'_{\text{data},2}, c'_{\text{ctrl}}], \dots, [c'_{\text{data},n}, c'_{\text{ctrl}}])$$

Given c' , $\text{Decode}(c')$ first recovers each m_i using $\text{Decode}_{\text{data}}(c'_{\text{data}}) = m_i$. This isn't as trivial as simply unique decoding as it outputs a list of potential decodes but a simple check is to compute m' and check if it matches y' by checking against $h_{r'}(m') = y'$. This recovers m and is based on the correctness of the underlying data code.

Correctness

The general set of algorithms rely on the correctness of the underlying control and data error correcting codes however, under the specific code settings, that is, randomized and self-seeded, the correctness relies on specific properties of the hash family $\mathcal{H}$. In both cases, the primary idea is that regardless of message choice, it should be hard to come up with a message chunk m'_i to replace m_i such that $m' = (m_1, m_2, \dots, m'_i, \dots, m_n)$ and $m = (m_1, m_2, \dots, m_i, \dots, m_n)$ where $m \neq m'$ but $h_r(m) = h_r(m')$.

1. Self-Seeded Codes require CRHFs: Since the randomness is fixed before the message is picked, the scheme requires $\mathcal{H}$ to be a collision resistant hash function as it ensures that for a given hash function h_r , finding two messages such that $m \neq m'$ but $h_r(m) = h_r(m')$ is hard for a polynomial time adversary
2. Randomized Codes require Universal One Way Hash Functions (UOWHFs) which require OWFs: Since randomness is set after the message is picked, UOWHFs ensure that a polynomial time adversary cannot find m' such that $m \neq m'$ but $h_r(m) = h_r(m')$

Exploring the 2025 Paper

The biggest limitation of the 2024 work was that the constructions worked for sufficiently large linear codes, that is $q \gg 2$ but the gold standard, if such a thing exists for EDAC schemes, is binary error detection and correction. In essence, larger alphabets by the very nature of including a lot of symbols include a lot of information with every symbol which are, ultimately, encoded as bit strings. As such, the 2025 work by Jad Silbak and Daniel Wichs attempts to address this limitation, specifically for seeded codes. It should be noted that seeded codes are distinct from self-seeded / randomized codes explored in the previous work in that the seed in seeded codes is shared between both the encoding and decoding algorithm whereas in self-seeded / randomized codes, the seed is only used by the encoding algorithm.

Selective Security vs. Adaptive Security

The goal of the adversary is to come up with codeword v' such that $\Delta(v, v') \leq p$ for $v = \text{Encode}_s(m)$ for some message m and seed s such that v' does not decode to m . This, however, can

be split into different cases depending on the order depending on when the adversary sees s , before or after picking m .

Selective Security

Under the notion of selective security, the adversary picks m before seeing s and is given $\text{Encode}_s(m) = v$. The adversary then picks v' such that $\Delta(v, v') \leq p$ and the goal for the EDAC scheme remains as it was before:

1. Detection: if $v' = v$, v' decodes to m otherwise, if $v' \neq v$, v' decodes to $\perp$ with overwhelming probability
2. Correction: v' decodes to m with overwhelming probability

The construction presented in the paper creates a selectively secure seeded code using Learning with Errors (LWE).

Adaptive Security

Under the notion of selective security, the adversary picks m after seeing s . The adversary is then given $\text{Encode}_s(m) = v$. The adversary then picks v' such that $\Delta(v, v') \leq p$ and the goal for the EDAC scheme remains as it was before:

1. Detection: if $v' = v$, v' decodes to m otherwise, if $v' \neq v$, v' decodes to $\perp$ with overwhelming probability
2. Correction: v' decodes to m with overwhelming probability

The construction presented in the paper creates an adaptively secure seeded code using Dark Matter hash assumption.

Construction

For all the constructions that follow, scheme's algorithms are denoted by $(\text{Encode}_s, \text{Decode}_s)$.

Selectively Secure Error Detection (SSED)

Let $\mathcal{H} = \{h : \{0, 1\}^k \rightarrow \{0, 1\}^v\}$ be a family of T2CI hash functions. Let $\mathbf{G} \in \mathbb{F}_2^{v \times n}$ be a generator matrix for linear efficiently list-decodable codes such that maps messages to codewords as $\mathbf{G} : \{0, 1\}^v \rightarrow \{0, 1\}^n$ where $n = k + v$. Take $\mathbf{G}$ and turn it into the full rank matrix $\mathbf{A} \in \mathbb{F}_2^{n \times n}$ by completing the basis, thus $\mathbf{A} = [\mathbf{G} \ \mathbf{B}]^T$. The encoding algorithm, $\text{Encode}_s(m)$ is as follows:

$$c = (m, h_s(m))\mathbf{A} = m\mathbf{B} + h_s(m)\mathbf{G}$$

The decoding algorithm, $\text{Decode}_s(c')$, interprets c as c' which does the following steps:

1. Compute $(m', y') = c'\mathbf{A}^{-1}$
2. Check $h_s(m') = y'$ and if true, output m' as m otherwise, output $\perp$

Selectively Secure Error Correction (SSEC)

The construction is almost the same, however, instead of $n = v + k$ in $\mathbf{G} \in \mathbb{F}_2^{v \times n}$, we have $n > v + k$ which means $\mathbf{A} \in \mathbb{F}_2^{(v+k) \times n}$ given $\mathbf{A}$ is defined as before, that is, $\mathbf{A} = [\mathbf{G} \ \mathbf{B}]^T$. Similarly, $\text{Encode}_s(m)$ remains the same so $c = m\mathbf{B} + h_s(m)\mathbf{G}$. The decoding algorithm $\text{Decode}_s(c')$ follows the following steps:

1. Applies list decoding over c' to obtain a list of (m', y') pairs
2. $\forall (m', y')$ pairs, check if $h_s(m') = y'$, if yes, then m' is the correct decoding otherwise output $\perp$

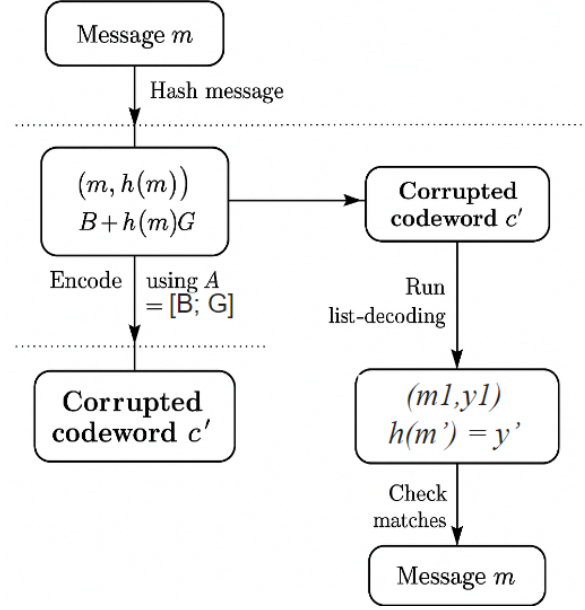


Figure 4: Selectively Secure Error Correction (SSEC)

Adaptively Secure Error Detection (ASED)

The authors construct an adaptively secure error detection seeded code under the Dark Matter hash assumption. The encoding algorithm $\text{Encode}_s(m)$ proceeds as follows:

1. Map m to $\tilde{m}$ via $\text{map}_{2 \rightarrow 3} : m \mapsto \tilde{m} \in \mathbb{F}_3^{\tilde{k}}$, where $\tilde{k} = \lceil (\log_3 2) \cdot k \rceil$.
2. Sample $\mathbf{B} \leftarrow \mathbb{F}_3^{\tilde{k} \times l}$ with $l = (1 + \delta) \cdot \tilde{k}$ for some small $\delta > 0$, and compute $y = \tilde{m} \cdot \mathbf{B}$.
3. Map y to $\tilde{y}$ via $\text{map}_{3 \rightarrow 2} : y \mapsto \tilde{y} \in \mathbb{F}_2^{\tilde{l}}$, where $\tilde{l} = \lceil (\log_2 3) \cdot l \rceil$.
4. Sample $\mathbf{A} \leftarrow \mathbb{F}_2^{\tilde{l} \times n}$ with $n = (1 + \delta) \cdot \tilde{l}$ and compute the final codeword $c = \tilde{y} \cdot \mathbf{A}$.

In compact form, we have $\text{Encode}_s(m) = \text{map}_{3 \rightarrow 2}(\text{map}_{2 \rightarrow 3}(m) \cdot \mathbf{B}) \cdot \mathbf{A}$. The authors consider both $\mathbf{A}$ and $\mathbf{B}$ as part of the seed. The decoding algorithm $\text{Decode}_s(c')$ computes $m' = \text{Encode}_s^{-1}(c')$; if both matrices are invertible (which occurs with high probability), then $m' = m$, otherwise the decoder returns $\perp$.

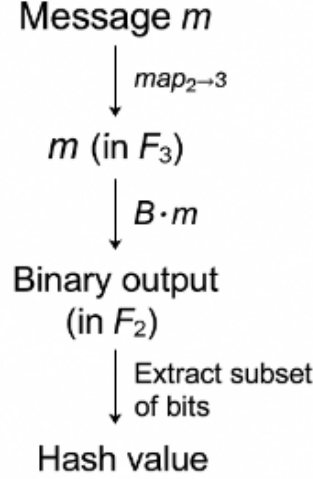


Figure 5: Adaptively Secure Error Detection (ASED)

Adaptively Secure Error Correction (ASEC)

Let $(\text{Encode}_{\text{List}}, \text{Decode}_{\text{List}})$ be an efficient list-encoding error-correcting code, and let $(\text{Encode}'_s, \text{Decode}'_s)$ be an adaptively secure error detection code. Then the encoding algorithm is defined as $\text{Encode}_s(m) = \text{Encode}_{\text{List}}(\text{Encode}'_s(m))$.

Since list decoding is used, $\text{Decode}_s(c')$ computes a list of candidate vectors v . If there exists a v such that $\text{Decode}'_s(v) = m' \neq \perp$, then m' is the correct message. If no valid decoding is found, the algorithm outputs $\perp$.

Correctness

1. **SSD and SSEC.** When there are no errors in the received codeword, decoding succeeds via simple matrix inversion. If the received codeword has errors but the decoder does not output $\perp$, then it must be the case that $h_s(m) = h_s(m')$ for some $m \neq m'$, which implies:

$$h(m')\mathbf{G} = (m - m')\mathbf{B} + h(m)\mathbf{G} + e$$

This further implies $h(m') \in f(m, m', h(m))$, violating the T2CI assumption. Thus, if an adversary can find $m' \neq m$ that passes verification despite errors, the T2CI hash family is insecure. This argument extends to SSEC as well, in the context of error correction rather than detection.

2. **ASED and ASEC.** Since both constructions rely on the Dark Matter hash assumption, they can be analyzed together. ASEC additionally uses a list-decoding scheme, which is assumed to be correct. The authors conjecture that the probability of finding a collision with h_s is less than $\frac{2^{o(t)}}{2^t}$. If this conjecture holds, both error detection and correction follow by analogous reasoning to the SSD and SSEC cases.

Conclusion

In this survey, we study two papers by Jad Silbak and Daniel Wichs that explore error detection and correction schemes under the notion of a computationally bounded adversarial communication

channel. In the 2024 paper, the authors present constructions that use minimal assumptions, namely one-way functions and collision resistant hash functions in order to construct codes with large alphabets. The constructions include randomized and self-seeded codes where a seed s is used only by the encoding algorithm. While the constructions are interesting, the use of large alphabets is not ideal as the gold standard for EDAC schemes is to come up with binary codes, that is, codes with alphabets made up of two symbols. In their 2025, this primary limitation is addressed. Under two notions of security, adaptive and selective, they construct seeded codes, that is, codes where both the encoding and decoding algorithms share a seed s that use a binary alphabet using slightly more complicated assumptions, namely, the dark matter hash assumption.

Acknowledgments

This project report / survey was prepared as part of the final project for Prof. Rishab Goyal's COMP SCI 880: Topics in Theoretical Computer Science during the Spring 2025 semester. We are grateful to Prof. Goyal for his guidance, supervision, and for an engaging semester of cryptography. We would also like to thank Jad Silbak and Daniel Wichs for their interesting papers, which this survey sought to study.